

Using ONNX (Open Neural Network Exchange)

What is ONNX and How Does it Work?

ONNX (often spelled/pronounced Onyx) stands for **Open Neural Network Exchange**. It is an open format built to represent machine learning models.

The Problem it Solves

Machine learning models are typically trained in frameworks like PyTorch (which was used for your 1D CNN), TensorFlow, or Scikit-Learn. However, when you want to deploy these models to production environments—such as a mobile app (Android/iOS), a web browser, or specialized hardware—relying on the heavy, training-focused PyTorch library isn't ideal or sometimes even possible.

How it Works

1. **Computational Graph:** When you convert a model to ONNX, it traces the mathematical operations (like Convolutions, Matrix Multiplications, and Activations) performed by your model and represents them as a static **computational graph**.
2. **Framework Agnostic:** This graph is saved in a standardized `.onnx` file format. By adhering to this universal standard, the model no longer requires PyTorch to run.
3. **Inference Engines:** On the deployment side (e.g., in your `vigidroid` Android app), you use an ONNX Runtime (like ONNX Runtime Mobile). This runtime reads the `.onnx` file and efficiently executes the computational graph on the target device's CPU, GPU, or Neural Processing Unit (NPU).

In this repository, it seems a PyTorch 1D CNN (`ByteCNN`) was trained to classify files (perhaps malware vs. benign) by looking at byte sequences, and saved as `1dcnn/trained_model.pth`. You can convert this to `.onnx` so that the `vigidroid` Android app can run predictions directly on a phone without needing PyTorch.

How to Convert Your 1D CNN `.pth` to `.onnx`

To convert the internal 1D CNN (defined in `1dcnn/src/model/bytecnn.py`), we need to:

1. Initialize the `ByteCNN` model architecture.
2. Load the trained weights from `trained_model.pth`.
3. Create a **dummy input tensor** that mimics a real input file (`batch_size`, `sequence_length`) to trace the graph.
4. Export it using `torch.onnx.export`.

Since the `ByteCNN` model takes an input sequence of bytes (values 0-255), we use integers as the dummy input.

PyTorch to ONNX Conversion Script

You can run this script to generate `model.onnx`. Create a file named `convert_to_onnx.py` in your `1dcnn` folder and run it:

```

import torch
import sys
import os

# Ensure the src folder is in the Python path so we can import the model
sys.path.append(os.path.join(os.path.dirname(__file__), 'src'))
from model.bytecnn import ByteCNN

def convert_to_onnx(pth_path, onnx_path):
    # 1. Initialize the model architecture
    print("Initializing model...")
    model = ByteCNN(embed_dim=8, num_classes=2)

    # 2. Load the trained weights
    print(f"Loading weights from {pth_path}...")
    state_dict = torch.load(pth_path, map_location="cpu")

    # Sometimes saved weights are inside a dict key (e.g.
    # 'model_state_dict')
    # If the file contains raw weights, this will work. Adjust if
    # necessary.
    if 'model_state_dict' in state_dict:
        model.load_state_dict(state_dict['model_state_dict'])
    else:
        model.load_state_dict(state_dict)

    # Set the model to inference mode (crucial for BatchNorm and Dropout
    # layers)
    model.eval()

    # 3. Create a dummy input tensor
    # The model expects a sequence of byte values (0-255).
    # Let's assume a dummy sequence length of 10,000 bytes for tracing.
    # The batch size is 1. (1, 10000)
    batch_size = 1
    sequence_length = 10000
    dummy_input = torch.randint(0, 256, (batch_size, sequence_length),
                                dtype=torch.long)

    # 4. Export the model to ONNX
    print(f"Exporting to {onnx_path}...")
    torch.onnx.export(
        model,                                # model being run
        dummy_input,                          # model input (or a tuple for
multiple inputs)
        onnx_path,                            # where to save the model (can be a
file or file-like object)
        export_params=True,                  # store the trained parameter weights
inside the model file
        opset_version=14,                    # the ONNX version to export the
model to
        do_constant_folding=True,           # whether to execute constant folding

```

```

for optimization
    input_names=['input_bytes'], # the model's input names
    output_names=['output'],     # the model's output names
    dynamic_axes={                # variable length axes (allows
dynamic sequence length or batch size during inference)
        'input_bytes': {0: 'batch_size', 1: 'sequence_length'},
        'output': {0: 'batch_size'}
    }
)
print("ONNX conversion complete!")

if __name__ == "__main__":
    pth_file = "trained_model.pth"
    onnx_file = "trained_model.onnx"
    convert_to_onnx(pth_file, onnx_file)

```

Steps to Run:

1. Make sure you are in a Python environment where `torch` is installed.
2. Save the code above as `1dcnn/convert_to_onnx.py`.
3. In your terminal, navigate to the `1dcnn` directory:

```
cd 1dcnn
```

4. Run the script:

```
python convert_to_onnx.py
```

5. You will now have a `trained_model.onnx` file! You can place this file into the Android project's `app/src/main/assets/` directory (like the existing `mh1m_2500_rp_XGBoost.onnx`) and load it onto the device using the ONNX Runtime for Java/Android.

1) Maintaining the same dimensions and parameters for ONNX conversion?

Yes, the conversion explicitly matches the PyTorch structure.

- **Architecture Parameters:** By default, `ByteCNN` is initialized with `embed_dim=8` and `num_classes=2`. The translation to ONNX will construct this exact same network architecture.
- **Input Dimensions:** As seen in `dataset.py` and `main.py`, the training script uses `BYTE_LENGTH = 1024`. Therefore, the ONNX model is exported using a dummy input tensor of shape `[1, 1024]` (Batch Size=1, Sequence Length=1024), and expects integer values ranging from `0` to `255` (representing byte values). The input data type will explicitly be `torch.long` (which translates to 64-bit integers in ONNX).
- **Padding/Truncation behavior:** In `dataset.py`, if the file is smaller than 1024 bytes, it is zero-padded. The exact same behavior will be replicated in the Java implementation before feeding the data to the ONNX model.

2) Side-by-Side Comparison: XGBoost vs 1D-CNN in `ScanService.java`

Here is exactly how the workflows compare at every stage of the pipeline within `ScanService.java`:

Pipeline Stage	Existing: XGBoost Model	Proposed: 1D-CNN Model
1. Model Loading / Initialization	Reads <code>mh1m_2500_rp_XGBoost.onnx</code> from <code>assets/</code> into an ONNX <code>OrtSession</code> . Also reads 2500 feature columns from a gzipped JSON file.	Reads the exported <code>1dcnn_model.onnx</code> from <code>assets/</code> into a <i>second</i> ONNX <code>OrtSession</code> . No external feature dictionaries are needed.
2. APK Parsing (Data Extraction)	Deep inspection: Opens <code>.apk</code> as a ZIP archive. Uses <code>AxmlReader</code> to decode <code>AndroidManifest.xml</code> and <code>MinimalDexParser</code> to parse <code>.dex</code> files. Extracts API calls, permissions, and intents as string tokens.	Shallow inspection: Treats the <code>.apk</code> purely as a binary file. Jumps to the very end of the file and reads strictly the last 1024 bytes .
3. Vectorization (Data Prep)	Iterates over the ~2500 known features. If a parsed token matches a feature, it assigns <code>1.0f</code> , else <code>0.0f</code> . Creates a <code>float[]</code> of size 2500.	Takes the extracted 1024 bytes and converts them into a <code>long[]</code> array. Due to Java's signed bytes, masks them with <code>& 0xFF</code> to ensure positive values (0-255). Zero-pads if file < 1024 bytes.
4. Submitting to ONNX Engine	Wraps the <code>float[]</code> in a <code>FloatBuffer</code> . Shape is passed as <code>[1, 2500]</code> to the ONNX Tensor.	Wraps the <code>long[]</code> in a <code>LongBuffer</code> . Shape is passed as <code>[1, 1024]</code> to the ONNX Tensor.
5. Inference Output	The ONNX runtime outputs a single float array, representing the direct malware probability score (e.g., <code>0.95</code>).	The ONNX runtime outputs raw logits of shape <code>[1, 2]</code> (e.g., <code>[Class 0 score, Class 1 score]</code>). The Java code will extract <code>Class 1</code> (Malware) and optionally apply a Softmax equation to get a percentage score.
6. Logging Performance	Logs <code>vecMs</code> (vectorization time) and <code>inferMs</code> (inference time) for the XGBoost model in a standard string.	Will compute and add <code>cnnParseMs</code> , <code>cnnInferMs</code> , and <code>cnnScore</code> to the exact same log broadcast, allowing you to directly compare the speeds and scores of both models.

By doing this, whenever a file is dropped in your `Downloads` folder, `ScanService` will run both algorithms back-to-back, giving you execution times, memory usage, and detection scores for both approaches simultaneously!

I have completed Phase 2. `ScanService.java` has been successfully updated to initialize, run, and log the results of both the existing XGBoost engine and the new 1D-CNN ONNX model side-by-side.

Here are the specific changes made to `ScanService.java`:

1. **Added `ortSessionCnn` field:** Added a second ONNX session instance.
2. **Updated `initOnnxModel()`:** Added I/O extraction code that copies the `1dcnn_model.onnx` from Android assets, caches it onto local storage, and initializes `ortSessionCnn` successfully.
3. **Implemented data extraction (`extractLastBytes`):** Added a helper that opens the `apk` file via `RandomAccessFile`, jumping straight to the end to grab the final 1024 bytes. It zero-pads left if the file is smaller than 1024 bytes. The raw bytes are also correctly handled with `& 0xFF` so they are strictly positive 0-255 integers explicitly requested by PyTorch (`torch.long`).
4. **Implemented inference (`runCnnInference`):** Wrapped the padded byte values in a `LongBuffer` and executed ONNX. To keep the score meaning identical to XGBoost (which returns a `float` measuring maliciousness probability), the outputs (model logits) are processed continuously alongside a Softmax equation returning exactly the scaled confidence of the `Malware` class.
5. **Combined Execution (`onHandleWork`):** Spliced this into the same work loop directly underneath the XGBoost execution metrics.
6. **Comparison logging:** Reconstructed `String result` output to broadcast parsing times, inference times, and scores of both engines to the Android UI. Wait intervals are measured accurately.
7. **Leak Protection:** Cleaned up the memory within `onDestroy()` by closing `ortSessionCnn`.

You can now build and test the Android APK using Android Studio to visualize how the dual-engine scan process works! Is there anything else you would like assistance with relating to this pipeline evaluations or fixes on?

Made changes.